

Q1. Create a 1D array of even numbers from 2 to 20 (inclusive). hint: Use np.max() and np.argmax

```
import numpy as np
arr=np.arange(2,21,2)
print("Array:",arr)
print("Max value:",np.max(arr))
print("Index of max value:",np.argmax(arr))
```

Array: [2 4 6 8 10 12 14 16 18 20]
Max value: 20
Index of max value: 9

Q2. Create a 5x5 identity matrix and print it. Use .reshape()

```
import numpy as np
arr=np.arange(25).reshape(5,5)
identity_matrix=np.eye(5)
print("5x5 Identity Matrix:")
print(identity_matrix)
```

5x5 Identity Matrix:
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]

Q3. Given arr = np.array([3, 7, 1, 9, 2, 5, 8, 4]), find the maximum value and its index without using Python's built-in max()

```
import numpy as np
arr=np.array([3,7,1,9,2,5,8,4])
max_value=np.max(arr)
max_index=np.argmax(arr)
print("Maximum value:",max_value)
print("Index of maximum value:",max_index)
```

Maximum value: 9
Index of maximum value: 3

Q4. Reshape the array np.arange(24) into a 3D array of shape (2, 3, 4). hint: Use .reshape()

```
import numpy as np
arr=np.arange(24)
arr_3d=arr.reshape(2,3,4)
print(arr_3d)
```

[[[0 1 2 3]
 [4 5 6 7]
 [8 9 10 11]]

 [[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]]

Q5. Given m = np.array([[4,9,2],[7,1,5],[3,8,6]]), extract all elements greater than 5 using boolean indexing.) hint: Create a mask

```
import numpy as np
m=np.array([[4,9,2],
            [7,1,5],
            [3,8,6]])
mask=m > 5
result=m[mask]
print("Boolean Mask:")
print(mask)
print("Elements greater than 5:")
print(result)
```

Boolean Mask:
[[False True False]
 [True False False]
 [False True True]]
Elements greater than 5:
[9 7 8 6]

Q6. Create two 3x3 random matrices A and B, compute their dot product, and print the trace (sum of diagonal elements). hint: Use np.random.rand()

```
import numpy as np
A = np.random.rand(3, 3)
B = np.random.rand(3, 3)
C = np.dot(A, B)
trace_value = np.trace(C)
print("Matrix A:")
print(A)
print("\nMatrix B:")
print(B)
print("\nDot Product of A and B:")
print(C)
print("\nTrace of the Dot Product Matrix:")
print(trace_value)
```

```
Matrix A:
[[0.71893961 0.98865513 0.90713553]
 [0.90251319 0.00258447 0.3504449 ]
 [0.02005499 0.50412706 0.94049706]]
```

```
Matrix B:
[[0.05880673 0.0898137 0.93830163]
 [0.38052225 0.62289761 0.69018345]
 [0.74145698 0.14954579 0.46016711]]
```

```
Dot Product of A and B:
[[1.09108573 0.81605985 1.77436956]
 [0.31389711 0.13507546 1.00987657]
 [0.89034904 0.45646813 0.7995436 ]]
```

```
Trace of the Dot Product Matrix:
2.025704794316101
```

Q7. Normalize the array `arr = np.array([10, 20, 30, 40, 50])` so that values range from 0 to 1. hint:Formula: $(arr - arr.min()) / (arr.max() - arr.min())$

```
import numpy as np
arr=np.array([10,20,30,40,50])
normalized_arr=(arr-arr.min())/(arr.max()-arr.min())
print("Normalized Array:")
print(normalized_arr)
```

```
Normalized Array:
[0.    0.25 0.5  0.75 1. ]
```

Q8. Stack two arrays `a = np.array([1,2,3])` and `b = np.array([4,5,6])` vertically and horizontally. hint:Use `np.vstack()` and `np.hstack()`

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
vertical_stack = np.vstack((a, b))
horizontal_stack = np.hstack((a, b))
print("Vertical Stack:")
print(vertical_stack)
print("\nHorizontal Stack:")
print(horizontal_stack)
```

```
Vertical Stack:
[[1 2 3]
 [4 5 6]]

Horizontal Stack:
[1 2 3 4 5 6]
```

